
Path Testing

Path testing

We need, first of all, to talk about path testing in general.

Program execution can follow distinct paths

Path testing is concerned with test cases that cause chosen paths

It's primarily applicable to component testing

It requires intimate knowledge of the program structure i.e. source code.

Path testing

When a program is executed it can follow various “paths”

Certain sequences of program code are executed depending on various criteria (e.g. value of data input to the program)

On any possible path through the system, the behavior may change in the sense that the execution of a particular statement is dependent on the statements that were executed prior to that and therefore influence the possible outcome of the statement execution

In path testing we are concerned with identifying possible paths through code and identifying test cases with knowledge of the possible paths in advance

Helps to raise our confidence in a piece of code because the more possible paths we have tested the more certain we can be about the behaviour of the system.

Bug assumption for path testing

We assume the program either:

- Makes the right decision at the wrong time OR

- Makes the wrong decision at the right time

That is, we assume the program takes the *wrong path*

Here are some code examples that might be defective:

- IF X>0 THEN... rather than IF X<0 THEN...

- IF SALARY > 100.00 rather than IF SALARY >=100.0

- GOTO A rather than GOTO B

- And so on...

Coverage

The two techniques that we need to discuss are **statement testing** and **branch testing**. Firstly, let's look at the coverage measurement.

Statement coverage

- most basic

- every statement executed at least once**

Branch coverage

- more refined

- every outcome of every decision executed at least once**

Coverage measurement

Achieving coverage is incremental

Coverage is normally measured as a percentage.

Statement Coverage = (statements executed / total statements) * 100

Branch Coverage = (branch outcomes executed / total branch outcomes) * 100

Path Coverage = (paths executed / total number of paths) * 100

Path Coverage in practice?

- Statement coverage is the easiest path testing to achieve.
- Branch / Decision coverage is more difficult, it requires testing all possible outcomes for all decisions/branches.
- Path coverages is the most difficult to achieve, as it is concerned with testing all possible outcomes from all possible statements / decisions and in all possible combinations. But is it necessary?

Procedure for statement testing

A generic process for statement test design.

- Identify all the executable statements

- Trace execution from first statement

- For each decision encountered, choose the true outcome first; write down values for the variable(s) in the predicates that make it true

- Are all statements covered? If not, select test values to reach the next uncovered statement in the code and continue covering statements as you proceed

- Repeat 4 until all statements covered by at least one test case

- For all test cases, record the variable values required to force the execution path you take.

From paths to test cases

Need to understand the relationship between paths and test cases

Process of choosing input values is called "sensitising the path"

Sometimes difficult for code buried deep in complex programs

You are free to choose which paths so choose the easiest.

Statement Testing

What are statements?

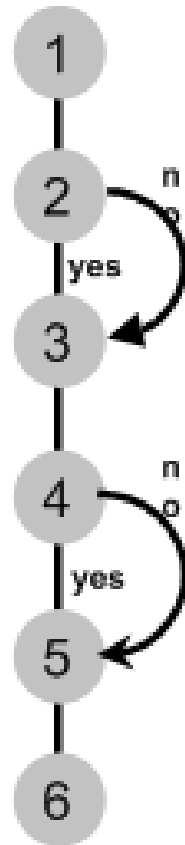
For the purpose of statement testing, executable statements “do something”

Data definitions, ELSEs, ENDIFs etc. do not count as executable statements

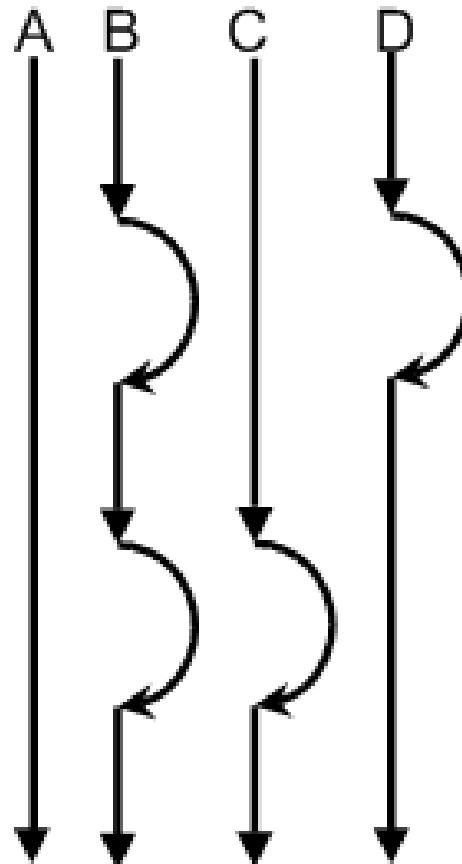
Statement coverage requires you to exercise each executable statement at least ONCE.

Sensitising the paths

Flow Graph



Four Possible Paths



- Two test cases are enough to cover all links in the flowgraph (A+B or C+D)
- Each path covers two conditions.

Statement coverage

(Statement coverage = No of statements Executed/Total no of statements in the source code * 100)

Example:

```
Read A
Read B
if A > B
    Print "A is greater than B"
else
    Print "B is greater than A"
endif
```

Set 1: If A = 5, B = 2

No of statements Executed: 5
Total no of statements in the source code: 7
Statement coverage = $5/7 \times 100 = 71.00\%$

Set 2: If A = 2, B = 5

No of statements Executed: 6
Total no of statements in the source code: 7
Statement coverage = $6/7 \times 100 = 85.20\%$

Pencils to the ready...

```
Read P
Read Q
IF P+Q > 100 THEN
Print "Large"
ENDIF
If P > 50 THEN
Print "P Large"
ENDIF
```

Statement Testing:

The test case(s) is / are executed in such a way that every statement of the code is executed at least once.

If a single path can execute all statements then choose this one – the most cost effective from a statement testing perspective..

Pencils to the ready...

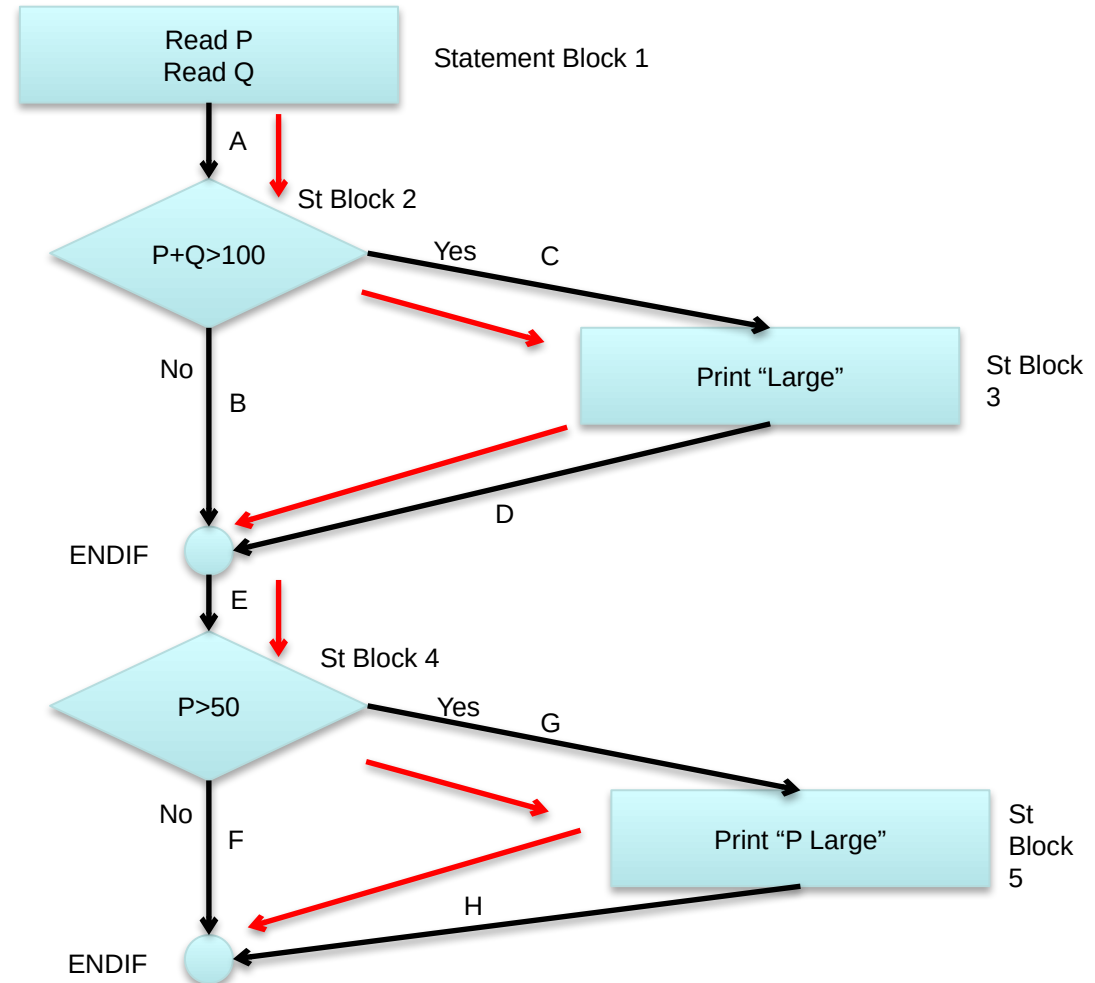
```
Read P
Read Q
IF P+Q > 100 THEN
  Print "Large"
ENDIF
If P > 50 THEN
  Print "P Large"
ENDIF
```

Statement Testing:

The test case(s) is / are executed in such a way that every statement of the code is executed at least once.

If a single path can execute all statements then choose this one – the most cost effective from a statement testing perspective..

Here by traversing through path 1A-2C-3D-E-4G-5H all the nodes are covered.



Branch/Decision Testing

Decision testing

Decision coverage requires you to **exercise all outcomes of the decisions** in the code

Normally, **reuse the test cases for statement testing and add new ones** to achieve this

From scratch, trace one path through the code, then pick off the remaining decisions with new tests.

Pencils to the ready... again!

```
Read P
Read Q
IF P+Q > 100 THEN
Print "Large"
ENDIF
If P > 50 THEN
Print "P Large"
ENDIF
```

Branch/Decision Testing:

The test case(s) is / are executed in such a manner so as to cover all possible true/false decisions.

Find out the minimum number of paths which will ensure covering of all the branches. In this case there is no single path which will ensure coverage of all the edges at one go..

Pencils to the ready... again!

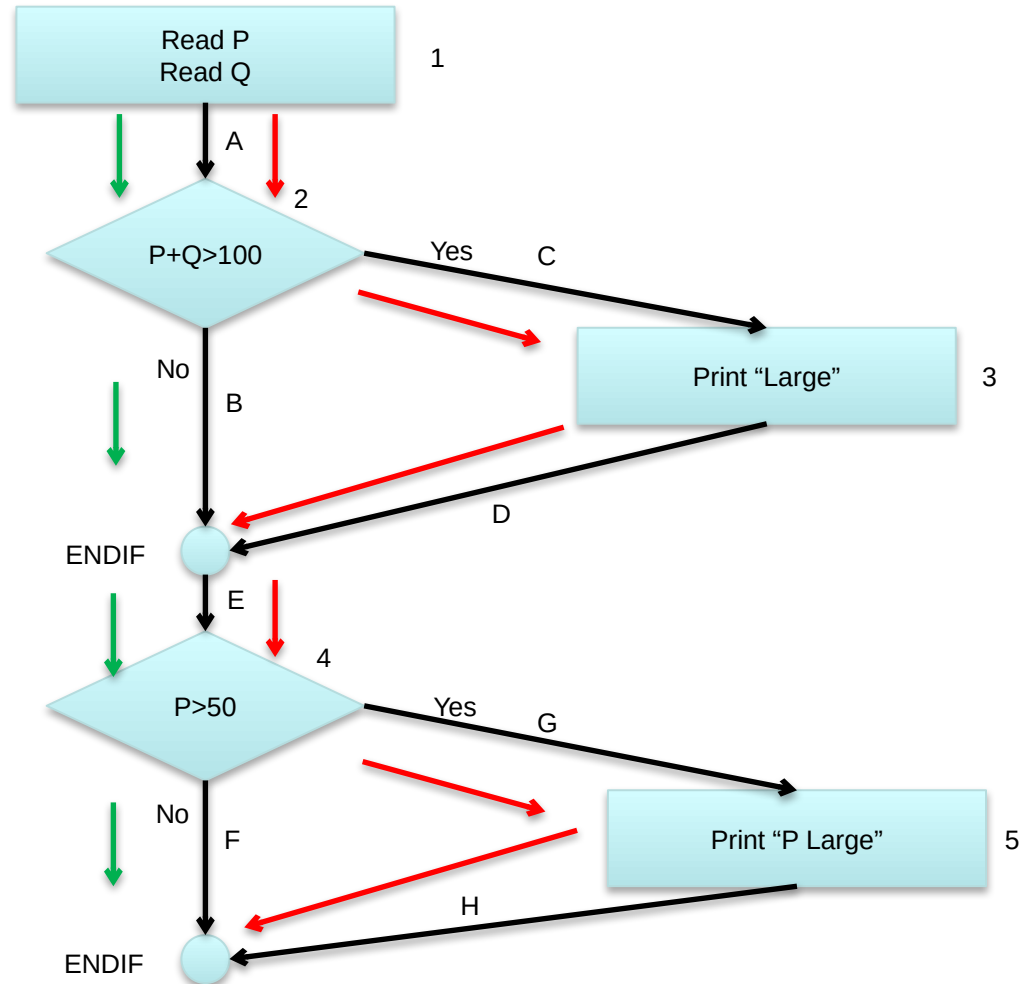
```
Read P
Read Q
IF P+Q > 100 THEN
  Print "Large"
ENDIF
If P > 50 THEN
  Print "P Large"
ENDIF
```

Branch/Decision Testing:

The test case(s) is / are executed in such a manner so as to cover all possible true/false decisions.

Find out the minimum number of paths which will ensure covering of all the branches. In this case there is no single path which will ensure coverage of all the edges at one go..

By following path **1A-2C-3D-E-4G-5H**, maximum numbers of edges (A, C, D, E, G and H) are covered but edges B and F are left. To cover these edges we can follow **1A-2B-E-4F**.



A note on path coverage

```
Read P
Read Q
IF P+Q > 100 THEN
  Print "Large"
ENDIF
If P > 50 THEN
  Print "P Large"
ENDIF
```

Path Coverage:

Path Coverage ensures covering of all the paths from start to end.

All possible paths are?

```
1A-2B-E-4F
1A-2B-E-4G-5H
1A-2C-3D-E-4G-5H
1A-2C-3D-E-4F
```

Hence **4 possible paths**.

